Name: Vivek Vaidya

Class: B1

PRN: 202501110081

1. Call Log Management — Singly Linked List

```
#include<iostream>

#include<string>

using namespace std;


struct Node

{

    string phone;

    string name;

    int duration;

    Node* next;

};


Node* head = NULL;


void addCall()

{

    Node* newNode = new Node;
```

```
cout << "Enter caller name: ";

cin >> newNode->name;


cout << "Enter phone number: ";

cin >> newNode->phone;


cout << "Enter call duration (minutes): ";

cin >> newNode->duration;


newNode->next = NULL;


if (head == NULL)

{

    head = newNode;

}

else

{

    Node* temp = head;


    while (temp->next != NULL)

    {

        temp = temp->next;

    }


    temp->next = newNode;

}
```

```
    cout << "Call Added Successfully\n";
}

void displayCalls()
{
    if (head == NULL)
    {
        cout << "Call Log is Empty\n";
        return;
    }

    Node* temp = head;

    while (temp != NULL)
    {
        cout << "\nCaller Name: " << temp->name;
        cout << "\nPhone Number: " << temp->phone;
        cout << "\nCall Duration: " << temp->duration << " minutes\n";

        temp = temp->next;
    }
}

void searchCall()
{
```

```
string searchNumber;


cout << "Enter phone number to search: ";

cin >> searchNumber;


Node* temp = head;


while (temp != NULL)

{

    if (temp->phone == searchNumber)

    {

        cout << "\nCall Found\n";

        cout << "Caller Name: " << temp->name << endl;

        cout << "Phone Number: " << temp->phone << endl;

        cout << "Call Duration: " << temp->duration << " minutes\n";

        return;

    }


    temp = temp->next;

}


    cout << "Call Not Found\n";

}


void deleteCall()

{
```

```
string searchNumber;


cout << "Enter phone number to delete: ";

cin >> searchNumber;


if (head == NULL)

{

    cout << "Call Log Empty\n";

    return;

}


if (head->phone == searchNumber)

{

    Node* temp = head;

    head = head->next;

    delete temp;


    cout << "Call Deleted\n";

    return;

}


Node* temp = head;


while (temp->next != NULL)

{

    if (temp->next->phone == searchNumber)
```

```
{
    Node* deleteNode = temp->next;
    temp->next = deleteNode->next;
    delete deleteNode;
        cout << "Call Deleted\n";
        return;
    }

    temp = temp->next;
}

cout << "Call Not Found\n";
}

int main()
{
    int choice;

    do
    {
    cout << "\n===== CALL LOG MANAGEMENT =====\n";
    cout << "1. Add Call\n";
    cout << "2. Display Calls\n";
    cout << "3. Search Call\n";
    cout << "4. Delete Call\n";
```

```
cout << "5. Exit\n";
cout << "Enter choice: ";
cin >> choice;

switch (choice)
{
   case 1:
      addCall();
      break;

   case 2:
      displayCalls();
      break;

   case 3:
      searchCall();
      break;

   case 4:
      deleteCall();
      break;

   case 5:
      cout << "Program Ended\n";
      break;
```

```
        default:
            cout << "Invalid Choice\n";
    }

} while (choice != 5);

    return 0;
}
```

Welcome | list1.cpp | list3.cpp | list2.cpp

list1.cpp > addCall()

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Node
{
    string phone;
    string name;
    int duration;
    Node* next;
};

Node* head = NULL;

void addCall()
{
    Node* newNode = new Node;

    cout << "Enter caller name: ";
    cin >> newNode->name;

    cout << "Enter phone number: ";
    cin >> newNode->phone;

    cout << "Enter call duration (minutes): ";
    cin >> newNode->duration;

    newNode->next = NULL;
```

Problems | Output | Debug Console | Terminal | Ports

C:\Users\Admin\Desktop\link list\list1.exe

```
PS C:\Users\Admin\Desktop\link list> & "C:\Users\Admin\Desktop\link list\list1.exe"
PS C:\Users\Admin\Desktop\link list> & "C:\Users\Admin\Desktop\link list\list1.exe"

===== CALL LOG MANAGEMENT =====
1. Add Call
2. Display Calls
3. Search Call
4. Delete Call
5. Exit
Enter choice: 1
Enter caller name: yasah
Enter phone number: 1234567890
Enter call duration (minutes): 45
Call Added Successfully

===== CALL LOG MANAGEMENT =====
1. Add Call
2. Display Calls
```

2. Train Reservation System — Doubly Linked List

```
#include<iostream>

#include<string>

using namespace std;


struct Node

{

    int PNR;

    string name;

    int age;

    int trainNumber;

    int seatNumber;


    Node* prev;

    Node* next;

};


Node* head = NULL;

Node* tail = NULL;


void addReservation()

{

    Node* newNode = new Node;
```

```
cout << "Enter PNR: ";

cin >> newNode->PNR;


cout << "Enter passenger name: ";

cin >> newNode->name;


cout << "Enter age: ";

cin >> newNode->age;


cout << "Enter train number: ";

cin >> newNode->trainNumber;


cout << "Enter seat number: ";

cin >> newNode->seatNumber;


newNode->next = NULL;

newNode->prev = NULL;


if (head == NULL)

{

    head = newNode;

    tail = newNode;

}

else

{

    tail->next = newNode;
```

```
    newNode->prev = tail;
    tail = newNode;
}

cout << "Reservation Added\n";
}

void displayForward()
{
    if (head == NULL)
    {
        cout << "No Reservations\n";
        return;
    }

Node* temp = head;

while (temp != NULL)
{
    cout << "\nPNR:"<<temp->PNR;
    cout << "\nPassenger Name:" << temp->name;
    cout << "\nAge:"<<temp->age;
    cout << "\nTrain Number:" << temp->trainNumber;
    cout << "\nSeat Number:" << temp->seatNumber << endl;

temp = temp->next;
```

```
    }
}

void displayBackward()
{
if (tail == NULL)
{
    cout << "No Reservations\n";
    return;
}

Node* temp = tail;

while (temp != NULL)
{
    cout << "\nPNR:"<<temp->PNR;
    cout << "\nPassenger Name:" << temp->name;
    cout << "\nAge:"<<temp->age;
    cout << "\nTrain Number:" << temp->trainNumber;
    cout << "\nSeat Number:" << temp->seatNumber << endl;

        temp = temp->prev;
    }
}

void searchReservation()
```

```
{
    int searchPNR;

    cout << "Enter PNR to search: ";
    cin >> searchPNR;

    Node* temp = head;

    while (temp != NULL)
    {
        if (temp->PNR == searchPNR)
        {
            cout << "\nReservation Found\n";
            cout << "PNR: " << temp->PNR << endl;
            cout << "Passenger Name: " << temp->name << endl;
            cout << "Age: " << temp->age << endl;
            cout << "Train Number: " << temp->trainNumber << endl;
            cout << "Seat Number: " << temp->seatNumber << endl;
            return;
        }
        temp = temp->next;
    }

    cout << "Reservation Not Found\n";
}
```

```
void cancelReservation()
{
    int searchPNR;

    cout << "Enter PNR to cancel: ";
    cin >> searchPNR;

    Node* temp = head;

    while (temp != NULL && temp->PNR != searchPNR)
    {
        temp = temp->next;
    }

    if (temp == NULL)
    {
        cout << "Reservation Not Found\n";
        return;
    }

    if (temp == head && temp == tail)
    {
        head = NULL;
        tail = NULL;
    }
```

```
else if (temp == head)
{
    head = head->next;
    head->prev = NULL;
}
else if (temp == tail)
{
    tail = tail->prev;
    tail->next = NULL;
}
else
{
    temp->prev->next = temp->next;
    temp->next->prev = temp->prev;
}

    delete temp;

    cout << "Reservation Cancelled\n";
}

int main()
{
    int choice;

    do
```

```
{
    cout << "\n===== TRAIN RESERVATION SYSTEM =====\n";
    cout << "1. Add Reservation\n";
    cout << "2. Display Forward\n";
    cout << "3. Display Backward\n";
    cout << "4. Search Reservation\n";
    cout << "5. Cancel Reservation\n";
    cout << "6. Exit\n";
    cout << "Enter choice: ";
    cin >> choice;

    switch (choice)
    {
    case 1:
        addReservation();
        break;

    case 2:
        displayForward();
        break;

    case 3:
        displayBackward();
        break;

    case 4:
```

```
            searchReservation();
            break;

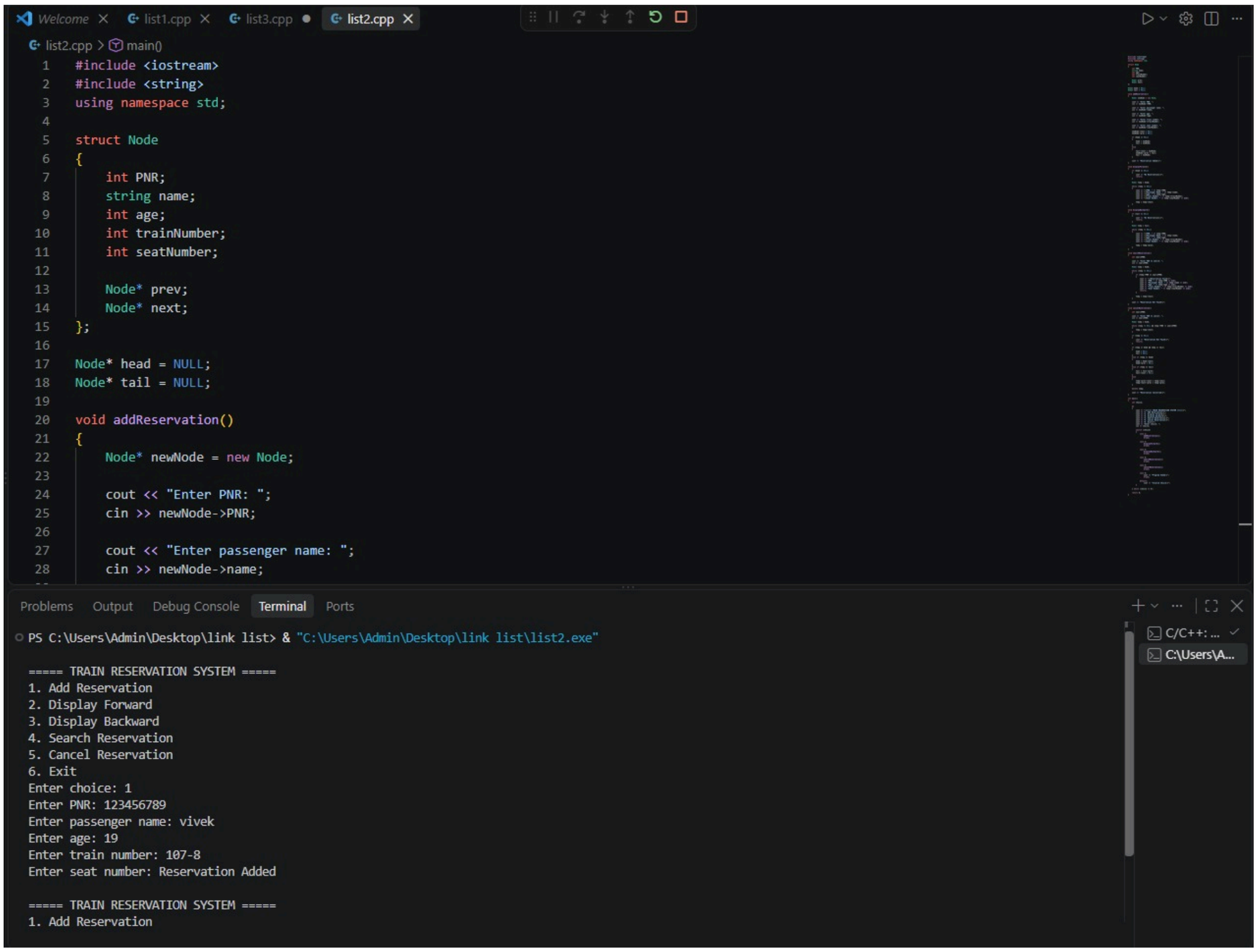
        case 5:
            cancelReservation();
            break;

        case 6:
            cout << "Program Ended\n";
            break;

        default:
            cout << "Invalid Choice\n";
        }

    } while (choice != 6);

    return 0;
}
```



3. Music Playlist Management — Circular Linked List

#include<iostream>

```
#include<string>
using namespace std;

struct Node
{
    int songID;
    string songName;
    string artist;
    float duration;

    Node* next;
};

Node* head = NULL;
Node* current = NULL;

void addSong()
{
    Node* newNode = new Node;

    cout << "Enter song ID: ";
    cin >> newNode->songID;

    cout << "Enter song name: ";
    cin >> newNode->songName;

    cout << "Enter artist name: ";
    cin >> newNode->artist;

    cout << "Enter duration (minutes): ";
    cin >> newNode->duration;

    if (head == NULL)
    {
        head = newNode;
        newNode->next = head;
```

```
    current = NULL;
}
else
{
    Node* temp = head;

    while (temp->next != head)
    {
        temp = temp->next;
    }

    temp->next = newNode;
    newNode->next = head;
  }

  cout << "Song Added\n";
}

void displayPlaylist()
{
  if (head == NULL)
  {
    cout << "Playlist Empty\n";
    return;
  }

  Node* temp = head;

  do
  {
    cout << "\nSong ID: " << temp->songID;
    cout << "\nSong Name: " << temp->songName;
    cout << "\nArtist: "<<temp->artist;
    cout << "\nDuration: "<<temp->duration<<" minutes\n";

    temp = temp->next;
```

```
    } while (temp != head);
}

void playNext()
{
    if (head == NULL)
    {
        cout << "Playlist Empty\n";
        return;
    }

    if (current == NULL)
    {
        current = head;
    }
    else
    {
        current = current->next;
    }

    cout << "\nNow Playing:\n";
    cout << "Song ID: " << current->songID << endl;
    cout << "Song Name: " << current->songName << endl;
    cout << "Artist: " << current->artist << endl;
    cout << "Duration: " << current->duration << " minutes\n";
}

void searchSong()
{
    if (head == NULL)
    {
        cout << "Playlist Empty\n";
        return;
    }
```

```
int searchID;

cout << "Enter song ID to search: ";
cin >> searchID;

Node* temp = head;

do
{
    if (temp->songID == searchID)
    {
        cout << "\nSong Found\n";
        cout << "Song ID: " << temp->songID << endl;
        cout << "Song Name: " << temp->songName << endl;
        cout << "Artist: " << temp->artist << endl;
        cout << "Duration: " << temp->duration << " minutes\n";
        return;
    }

    temp = temp->next;

} while (temp != head);

cout << "Song Not Found\n";
}

void deleteSong()
{
    if (head == NULL)
    {
        cout << "Playlist Empty\n";
        return;
    }

int searchID;
```

```
cout << "Enter song ID to delete: ";
cin >> searchID;

if (head->songID == searchID)
{
    if (head->next == head)
    {
        delete head;
        head = NULL;
        current = NULL;
    }
    else
    {
        Node* temp = head;

        while (temp->next != head)
        {
            temp = temp->next;
        }

        Node* deleteNode = head;
        head = head->next;
        temp->next = head;

        if (current == deleteNode)
        {
            current = NULL;
        }

        delete deleteNode;
    }

    cout << "Song Deleted\n";
    return;
}
```

```
Node* temp = head;

while (temp->next != head)
{
    if (temp->next->songID == searchID)
    {
        Node* deleteNode = temp->next;
        temp->next = deleteNode->next;

        if (current == deleteNode)
        {
            current = NULL;
        }

        delete deleteNode;

        cout << "Song Deleted\n";
        return;
    }

    temp = temp->next;
}

    cout << "Song Not Found\n";
}

int main()
{
    int choice;

    do
    {
        cout << "\n===== MUSIC PLAYLIST MANAGEMENT =====\n";
        cout << "1. Add Song\n";
        cout << "2. Display Playlist\n";
        cout << "3. Play Next Song\n";
```

```
cout << "4. Search Song\n";
cout << "5. Delete Song\n";
cout << "6. Exit\n";
cout << "Enter choice: ";
cin >> choice;

switch (choice)
{
   case 1:
      addSong();
      break;

   case 2:
      displayPlaylist();
      break;

   case 3:
      playNext();
      break;

   case 4:
      searchSong();
      break;

   case 5:
      deleteSong();
      break;

   case 6:
      cout << "Program Ended\n";
      break;

   default:
      cout << "Invalid Choice\n";
}
```

```
    } while (choice != 6);

    return 0;
}
```

